

Instrukcja skoku goto

1. Podstawy

Czasami istnieje konieczność przeniesienia się w inne miejsce algorytmu. Zaznaczamy takie miejsca w kodzie (do których chcemy się odwołać) poprzez wstawienie punktów kontrolnych. Obowiązują dokładnie takie same zasady co w nazewnictwie zmiennych – punkt kontrolny nie może być liczbą, nie może zaczynać się od cyfry, nazwa powinna sugerować co on ma robić, nazwa musi być jednym ciągiem. Za punktem kontrolnym stawiamy dwukropek. Punkt kontrolny jest „przezroczysty” z punktu wykonania kodu. Wykonując algorytm przechodzimy przez taką deklarację, jak gdyby nigdy nic. W całym algorytmie może być tylko jeden punkt kontrolny o takiej samej nazwie. Możemy używać tylu punktów kontrolnych ilu chcemy, ale każdy z nich musi mieć inną nazwę.

Punkty kontrolne działają dokładnie tak samo, jak te używane w zapisie Pseudojęzyka.

Aby przeskoczyć do miejsca kodu oznaczonego punktem kontrolnym używamy instrukcji skoku **goto** po czym podajemy nazwę punktu kontrolnego, do którego chcemy przeskoczyć.

W algorytmie możemy mieć wiele różnych skoków do tego samego punktu kontrolnego. Skakać możemy zarówno do tyłu jak i do przodu algorytmu.

Funkcjonalność przydatna np. do wykonywania pętli (pętle poznamy na kolejnych lekcjach).

Przykład. Algorytm drukuje tylko komunikaty „Elo” oraz „Eluwina”.

```
cout << "Elo!" << "\n";  
goto po_wydruku; // Tuśś średnik na końcu, skok do PK cout << "Hello!" << "\n";  
cout << "Czesc!" << "\n";  
cout << "Witaj!" << "\n";  
po_wydruku: // Tuśś dwukropek na końcu, deklaracja PK cout << "Eluwina!" << "\n";
```

2. Pętla realizowana instrukcją skoku

Konstrukcja algorytmu typu **pętla** to fragmenty algorytmu wykonujące się kilkakrotnie, dopóki spełnione są określone warunki. W pętli możemy wyróżnić (najczęściej) podstawowe elementy:

- licznik (w jakiejś formie) oraz nadanie mu wartości początkowej.
- warunek wykonania/stopu. Nazwa zależy od konstrukcji warunku. Jeśli warunek jest tak skonstruowany, że „prawda” powoduje kolejne wykonanie, natomiast „fałsz” przerwanie takiej pętli, to nazywamy go warunkiem wykonania. Jeśli warunek jest tak skonstruowany, że „fałsz” powoduje kolejne wykonanie, natomiast „prawda” przerwanie takiej pętli, to nazywamy go warunkiem stopu.

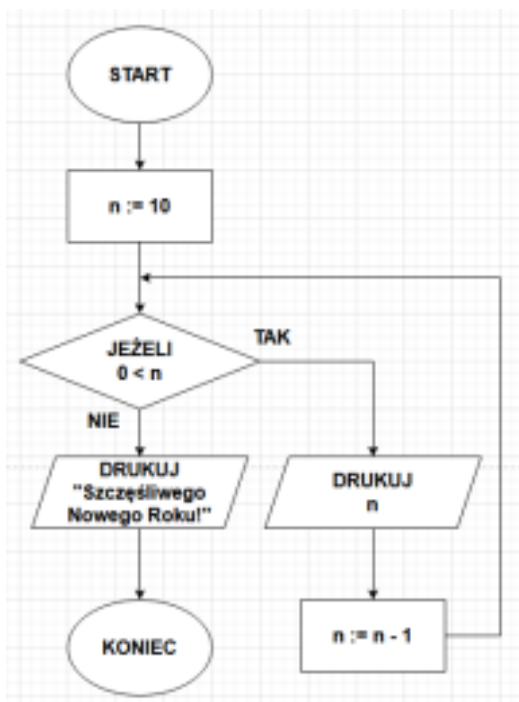
- modyfikator licznika (w jakiejś formie),

Czasami spotyka się pętle bez użycia licznika, z samym tylko warunkiem stopu/wykonania.

Uwaga! W profesjonalnym programowaniu używanie instrukcji **goto** jest odradzane ze względu na to, że takie przeskoki w dowolne miejsca programu mogą powodować, że będziemy m.in. gubić zmienne, a tym samym powodować wycieki pamięci. Na przykład, przejście kodu przez `}` usuwa z pamięci zmienne zadeklarowane w tym bloku kodu. Wskoczenie z bloku kodu poprzez **goto** pominięte ten krok (zmienne, chociaż już niepotrzebne, zostaną w pamięci).

Warto jednak wiedzieć, że taka instrukcja jest oraz w jaki sposób działa.

Przykładowy algorytm licznika odliczający sekundy do Nowego Roku:



```

int n = 10;

count:

if (0 < n)
{
    cout << n << "\n";
    n--;
    goto count;
}

cout << "Szczęśliwego Nowego Roku!";
  
```